

TRABAJO GRUPAL 1: MICROSERVICIOS CON QUARKUS, SPRING BOOT Y DESPLIEGUE KUBERNETES

PROGRAMACIÓN DISTRIBUIDA

Docente: Jaime Salvador M.

Fecha: 2026-07-09

Duración: 2 semanas, **Puntos:** 20 puntos (rúbrica adjunta)

Nombre:
Firma:

1. Objetivo

Implementar desde cero los **4 módulos** de la aplicación Books, partiendo de los ejemplos de proyecto (build files y configuraciones mínimas) desarrollados en clases. Cada módulo debe construirse en su totalidad: capa de persistencia, lógica de negocio, exposición REST, service discovery, health checks, métricas y despliegue en Kubernetes.

Servicio	Framework	Puerto	Estado actual
app-authors	Quarkus 3.37.2	8070	Por implementar
app-books	Quarkus 3.37.2	8080	Por implementar
app-customers	Spring Boot 4.1.0	8090	Por implementar
app-web	React 19 + Vite 8	5173	Por implementar

El grupo deberá construir y demostrar:

- **app-authors:** Microservicio Quarkus con entidad Author, persistencia PostgreSQL, CRUD completo, Flyway, registro en Consul, health checks con MicroProfile Health y métricas Prometheus.
- **app-books:** Microservicio Quarkus con entidades Book/Inventory, CRUD completo, cliente REST hacia app-authors con service discovery (Stork + Consul), circuit breaker (SmallRye Fault Tolerance), health checks con MicroProfile Health y métricas Prometheus.
- **app-customers:** Microservicio Spring Boot 4.1.0 con entidad Customer, CRUD completo, registro en Consul, health checks con Actuator y métricas Prometheus.
- **app-web:** Frontend React 19 + TypeScript 6 + Vite 8 con navegación, CRUD sobre las 3 entidades y estado global.
- Despliegue Kubernetes: Manifiestos completos para los 7 componentes (PostgreSQL, Consul, Traefik, app-authors, app-books, app-customers, app-web) sobre **minikube** y **OpenShift Developer Sandbox**.
- Health checks: Endpoints `/health` (MicroProfile Health) en los servicios Quarkus y `/actuator/health` en Spring Boot, con readiness/liveness probes en Kubernetes.
- Resiliencia: Circuit breaker en la comunicación entre app-books y app-authors.
- Monitoreo: Prometheus scraping de los 3 microservicios backend.

2. Stack tecnológico

2.1 Backend — app-authors, app-books (Quarkus 3.37.2)

Componente	Tecnología
Framework	Quarkus 3.37.2
Lenguaje	Java 25
REST	Jakarta REST (JAX-RS)
Persistencia	Hibernate ORM + Panache
Migraciones	Flyway (solo app-authors)
BD	PostgreSQL 17
Service Discovery	Consul 1.15 (vía Vert.x ConsulClient programático)
REST Client	MicroProfile REST Client + Stork (cuando aplique)

Componente	Tecnología
Health Checks	MicroProfile Health
Métricas	Micrometer + Prometheus
Fault Tolerance	SmallRye Fault Tolerance (@Retry, @CircuitBreaker, @Fallback)
Contenedores	Dockerfile
Build	Gradle 9.x con Kotlin DSL

2.2 Backend — app-customers (Spring Boot 4.1.0)

Componente	Tecnología
Framework	Spring Boot 4.1.0
Lenguaje	Java 25
REST	Spring WebMVC
Persistencia	Spring Data JPA
BD	PostgreSQL 17
Service Discovery	Spring Cloud Consul Discovery
Health Checks	Spring Boot Actuator
Métricas	Micrometer + Prometheus
Containerización	Dockerfile
Build	Gradle 9.x con Kotlin DSL

2.3 Frontend (app-web)

Componente	Tecnología
Framework	React 19
Lenguaje	TypeScript 6 (modo estricto)
Build	Vite 8
HTTP Client	axios
Routing	react-router-dom v7+
Estado global	Redux Toolkit o Zustand
Estilos	CSS Modules o Tailwind CSS (a elección)

2.4 Infraestructura

Componente	Tecnología
Orquestador local	minikube/kind
Orquestador remoto	OpenShift Developer Sandbox
Service Discovery	Consul 1.15
API Gateway	Traefik 3.x (con provider Consul Catalog)
Base de datos	PostgreSQL 17
Monitoreo	Prometheus + Grafana (opcional)

3. Requerimientos funcionales — app-authors (Quarkus)

3.1 Entidades y base de datos

Entidades Author (id, name) y BookAuthor (relación many-to-many con ISBN). Scripts Flyway con CREATE TABLE + datos de prueba (3 autores mínimo).

3.2 Endpoints REST

Método	Ruta	Comportamiento
GET	/authors	Listar todos (200 + List<Author>)
GET	/authors/{id}	Buscar por id (200 / 404). El nombre debe incluir el puerto del servidor.
GET	/authors/find/{isbn}	Autores de un libro por ISBN (200 + List<AuthorDto>) — incluir retry/failure simulado
POST	/authors	Crear (201 + recurso creado)
PUT	/authors/{id}	Actualizar (200 / 404)
DELETE	/authors/{id}	Eliminar (204 / 404)

3.3 Capa de persistencia

AuthorRepository extendiendo PanacheRepositoryBase<Author, Integer> con método findByBook(String isbn).

3.4 Service Discovery

Registro y desregistro programático en Consul (Vert.x ConsulClient) con tags Traefik (PathPrefix /app-authors, stripprefix). Health check HTTP contra /q/health/live.

3.5 Health Checks (MicroProfile Health)

DatabaseHealthCheck implementando HealthCheck con @Liveness y @Readiness que verifique la conexión a PostgreSQL.

3.6 Métricas y CORS

CORS para http://localhost:5173 y dominio OpenShift. Métricas expuestas en /q/metrics.

4. Requerimientos funcionales — app-books (Quarkus)

4.1 Entidades y base de datos

Entidades Book (isbn PK, title, price, version) e Inventory (OneToOne). Scripts Flyway con CREATE TABLE + datos de prueba.

4.2 Endpoints REST

Método	Ruta	Comportamiento
GET	/books	Listar todos con autores + inventario (200)
GET	/books/{isbn}	Buscar por ISBN con autores + inventario (200 / 404)
POST	/books	Crear (201 + Location header)
PUT	/books/{isbn}	Actualizar (200 / 404)
DELETE	/books/{isbn}	Eliminar (204 / 404)

BookDto debe incluir: isbn, title, price, authors, inventorySold, inventorySupplied.

4.3 Cliente REST hacia app-authors

Interfaz AuthorRestClient con MicroProfile REST Client (@RegisterRestClient apuntando a stork://authors-api).

4.4 Service Discovery

Stork configurado con Consul para descubrir authors-api.

4.5 Health Checks (MicroProfile Health)

- DatabaseHealthCheck — verifica conexión a BD.
- AuthorsServiceHealthCheck — verifica disponibilidad de app-authors. Si el circuito está abierto, reporta DOWN.

4.6 Resiliencia — Circuit Breaker

@CircuitBreaker (threshold=3, delay=5s) + @Retry + @Fallback en AuthorRestClient.findByBook.

4.7 Métricas y CORS

CORS para http://localhost:5173 y dominio OpenShift. Métricas en /q/metrics.

5. Requerimientos funcionales — app-customers (Spring Boot)

5.1 Entidad y base de datos

Entidad Customer (id, firstName, lastName, email único, phone, address, createdAt). Schema SQL con datos de prueba (5 clientes).

5.2 Endpoints REST

Método	Ruta	Comportamiento
GET	/customers	Listar todos (200)
GET	/customers?email={email}	Filtrar por email (200)
GET	/customers/{id}	Buscar por id (200 / 404)
POST	/customers	Crear (201). Validar email único: 400 si ya existe
PUT	/customers/{id}	Actualizar (200 / 404)
DELETE	/customers/{id}	Eliminar (204 / 404)
GET	/ping	"pong"

5.3 Capa de persistencia

CustomerRepository extendiendo JpaRepository<Customer, Integer> con findByEmail(String).

5.4 Service Discovery

Registro en Consul con Spring Cloud Consul Discovery. Tags Traefik (PathPrefix /app-customers, stripprefix).

5.5 Health Checks (Spring Boot Actuator)

Endpoints /actuator/health, /actuator/health/liveness, /actuator/health/readiness habilitados.

5.6 Métricas

Exponer /actuator/prometheus con Micrometer.

5.7 CORS

CORS para http://localhost:5173 y dominio OpenShift.

6. Requerimientos funcionales — app-web (React + TypeScript)

Construir desde cero una SPA con:

6.1 Navegación

Ruta	Vista
/	Home / Dashboard
/authors	CRUD Autores
/books	CRUD Libros (con autores)
/customers	CRUD Clientes

6.2 Funcionalidad CRUD

Para cada entidad: listar (tabla), crear (formulario), editar, eliminar (con confirmación), ver detalle.

6.3 Requerimientos técnicos

- React Router DOM v7+.
- Estado global (Redux Toolkit o Zustand) con manejo de loading y error.
- axios con instancia y manejo de errores centralizado (interceptors).
- TypeScript estricto con interfaces para todas las entidades.
- Separación en `services/`, `store/`, `components/`, `pages/`.
- Notificaciones visuales de error (toast/banner).
- Indicador de carga (spinner/skeleton).

7. Despliegue Kubernetes

Crear todos los manifiestos desde cero en `deployment/k8s/`. No se asume ningún manifiesto preexistente como correcto o completo.

7.1 Namespace

`deployment/k8s/namespace.yml`

Todos los recursos deben pertenecer a namespace: `books-app`.

7.2 ConfigMap y Secret

Los datos sensibles de conexión a la base de datos (username, password, URL) **no deben ir fijos** en los manifiestos de los Deployments. En su lugar:

- Crear un **ConfigMap** con valores no sensibles: URL de conexión (`jdbc:postgresql://postgres:5432/books-db`), puerto HTTP, host y puerto de Consul.
- Crear un **Secret** con valores sensibles: username (`postgres`) y password (`postgres`) de la BD.
- Cada Deployment debe referenciar estos recursos vía `envFrom` o `valueFrom/secretKeyRef` y `configMapKeyRef`.

Archivos a crear dentro de `deployment/k8s/`:

```
deployment/k8s/
|-- books-config.yml      (ConfigMap)
\-- books-secret.yml      (Secret)
```

7.3 Manifiestos requeridos

Archivo	Componente	Réplicas	Probes
postgresql-deployment.yml	PostgreSQL 17 (Deployment + PVC + Service)	1	—
consul-deployment.yml	Consul 1.15 (Deployment + Service)	1	—
traefik-deployment.yml	Traefik 3.x (Deployment + Service)	1	—
app-authors-deployment.yml	app-authors Quarkus (Deployment + Service)	2	Liveness, Readiness
app-books-deployment.yml	app-books Quarkus (Deployment + Service)	2	Liveness, Readiness
app-customers-deployment.yml	app-customers Spring Boot (Deployment + Service)	2	Liveness, Readiness
app-web-deployment.yml	app-web Nginx (Deployment + Service)	1	—
books-config.yml	ConfigMap con valores no sensibles	—	—
books-secret.yml	Secret con credenciales de BD	—	—
route.yml	Route para OpenShift	—	—

7.4 Variables de entorno en los Deployments

Cada Deployment de microservicio debe obtener las variables desde el ConfigMap y el Secret creados. Valores que debe contener cada recurso:

Variable	ConfigMap	Secret
QUARKUS_DATASOURCE_JDBC_URL / SPRING_DATASOURCE_URL	jdbc:postgresql://postgres:5432/books-db	—
QUARKUS_DATASOURCE_USERNAME / SPRING_DATASOURCE_USERNAME	—	postgres
QUARKUS_DATASOURCE_PASSWORD / SPRING_DATASOURCE_PASSWORD	—	postgres
QUARKUS_HTTP_PORT / SERVER_PORT	8080	—
CONSUL_HOST	consul	—
CONSUL_PORT	8500	—
QUARKUS_FLYWAY_MIGRATE_AT_START	true	—

8. Monitoreo con Prometheus

8.1 Configuración de scraping

Crear deployment/telemetria/prometheus-config.yml (ConfigMap) con targets:

8.2 Despliegue de Prometheus en Kubernetes

deployment/telemetria/prometheus-deployment.yml — ConfigMap + Deployment + Service.

9. Estructura de entregables

Subir a TEAMS un archivo ZIP con la siguiente estructura:

```
tarea-grupal/
|-- app-authors/
|-- app-books/
|-- app-customers/
```

```
|-- app-web/
|-- deployment/
\-- README.md
```

10. README.md

Debe contener:

1. **Integrantes:** nombres completos y cédulas.
2. **Arquitectura:** diagrama de bloques mostrando:
 - Los 3 microservicios backend, el frontend, Consul, Traefik, PostgreSQL.
 - Las relaciones de comunicación (REST síncrono, service discovery, API Gateway).
 - Los componentes Kubernetes (Pods, Services, Ingress/Route).
3. **Tabla de endpoints** de cada microservicio (método, ruta, código de respuesta).
4. **Health checks:** tabla con las URLs de health y el framework usado:
5. **Instrucciones de ejecución:**
 - **Local (docker-compose):** `docker-compose up --build` desde `deployment/docker-compose/`.
 - **Minikube:** comandos para `minikube start`, `kubectl apply -f deployment/k8s/`, `minikube service ...`
 - **OpenShift:** comandos para `oc apply -f deployment/k8s/`, `oc create route ...`
6. **Verificación:** ejemplos de `curl` para cada endpoint de los 3 servicios.
7. **Circuit Breaker:** explicación del comportamiento y cómo probarlo (detener `app-authors` y verificar fallback + health check DOWN).
8. **Métricas:** cómo acceder a Prometheus y consultas de ejemplo.
9. **Capturas de pantalla:**
 - Dashboard de Consul mostrando los 3 servicios registrados.
 - Página principal de la app web mostrando datos de Authors, Books y Customers.
 - Traefik Dashboard o respuesta de un endpoint.
 - Prometheus targets UP.
 - Salida de `curl /q/health/live` para cada servicio Quarkus.
 - Salida de `curl /actuator/health` para `app-customers`.
10. **Decisiones de diseño:** justificar framework de estado global, estructura de componentes, etc.

11. Criterios de evaluación (rúbrica — 20 puntos)

11.1 app-authors — Quarkus (4 puntos)

Criterio	Pts
Entidades JPA Author + BookAuthor, Flyway con datos de prueba	1
CRUD completo (GET list, GET by id, GET find/{isbn}, POST, PUT, DELETE)	1
Registro en Consul con tags Traefik + health check HTTP	1
Health checks MicroProfile Health + métricas Prometheus	1

11.2 app-books — Quarkus (5 puntos)

Criterio	Pts
Entidades JPA Book + Inventory, Flyway con datos de prueba	1
CRUD completo con BookDto (autores + inventario)	1
Cliente REST a app-authors con Stork + Consul	1
Circuit Breaker (@CircuitBreaker + @Retry + @Fallback)	1
Health checks MicroProfile Health (BD + AuthorsService) + métricas Prometheus	1

11.3 app-customers — Spring Boot (4 puntos)

Criterio	Pts
Entidad Customer JPA + validación email único	1
CRUD completo (GET list, GET by id, GET by email, POST, PUT, DELETE)	1
Registro en Consul con tags Traefik	1
Health checks Actuator (liveness + readiness) + métricas Prometheus	1

11.4 app-web — Frontend React (4 puntos)

Criterio	Pts
Navegación con React Router (3 rutas + home)	1
CRUD de Authors + Books + Customers funcionando (al menos 2 entidades completas)	1
Estado global (Redux/Zustand) con manejo de loading y error	1
Compila con <code>npm install && npm run build</code> ; se ejecuta con <code>npm run dev</code>	1

11.5 Despliegue Kubernetes (2 puntos)

Criterio	Pts
Manifiestos completos (namespace, ConfigMap, Secret, postgres, consul, traefik, 3 apps, web, ingress, route)	1
ConfigMap + Secret correctamente referenciados vía <code>envFrom</code> en los Deployments; Liveness + Readiness probes correctas	1

11.6 README (1 punto)

Criterio	Pts
README completo con diagrama de arquitectura, tabla de endpoints, health checks, instrucciones minikube/OpenShift, curl de verificación, capturas de Consul + app web + Prometheus	1

Total: 20 puntos**12. Restricciones**

- **No se permite** usar bases de datos embebidas (H2, HSQLDB). La persistencia debe ser contra PostgreSQL 17.
- **No se permite** el uso de frameworks frontend distintos a React + Vite + TypeScript.
- **No se permite** el uso de frameworks backend distintos a Quarkus 3.37.2 (app-authors, app-books) y Spring Boot 4.1.0 (app-customers).
- **Health checks obligatorios:** MicroProfile Health en los servicios Quarkus, Actuator probes en Spring Boot. No se aceptan endpoints /ping caseros como sustituto de los health checks formales.
- Cualquier copia entre grupos (o de soluciones encontradas en línea) será calificada con **0** (cero) y reportada según el reglamento.
- El proyecto debe entregarse **funcionando**.
- En caso de detectarse proyectos generados con IA Generativa, recibirán la nota de **0** (cero).